

Hierarchical models

Jure Demšar and Erik Štrumbelj, University of Ljubljana

/ Summary

In Bayesian modeling hierarchical models (also called multilevel models) are an extremely powerful tool. As already emphasized a couple of times now, when modeling we should try to describe the data generating process (and not fit some distributions to some data). Since data generating processes often have a hierarchical structure (e.g., groups of students, multiple repetitions of an experiment...), hierarchical models enable us to efficiently describe such data generating processes.

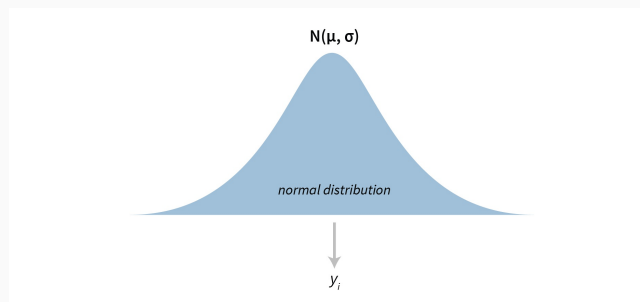
/ Piglets

We will build our first hierarchical (multilevel) model from the ground up by using a practical example. Our dataset (`./data/piglets.csv`) includes the weight of little piggies at birth along with the information about who the mother of each of the piglets is and at which farm they live. To get familiar with hierarchical models, we will initially focus only on pigs from the first farm. Let us use n to denote the number of piglets and m to denote the number of mama pigs. For the sake of this illustrative example we will assume that weights of piglets at birth are normally distributed.

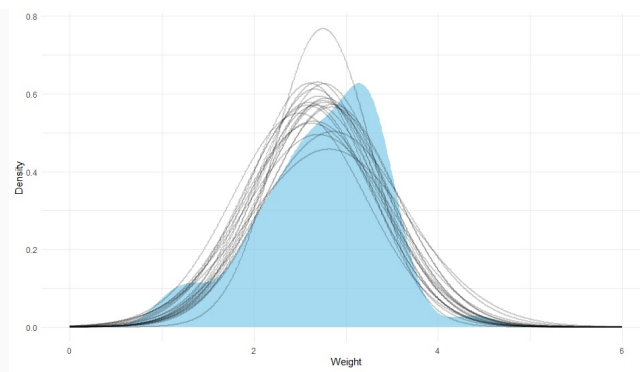
The simplest model we can build is the traditional normal model (`./models/normal.stan`):

$$y_i \mid \mu, \sigma \sim N(\mu, \sigma),$$

where y_i denotes the birth weight of piglet i . Visually this model can be represented as:



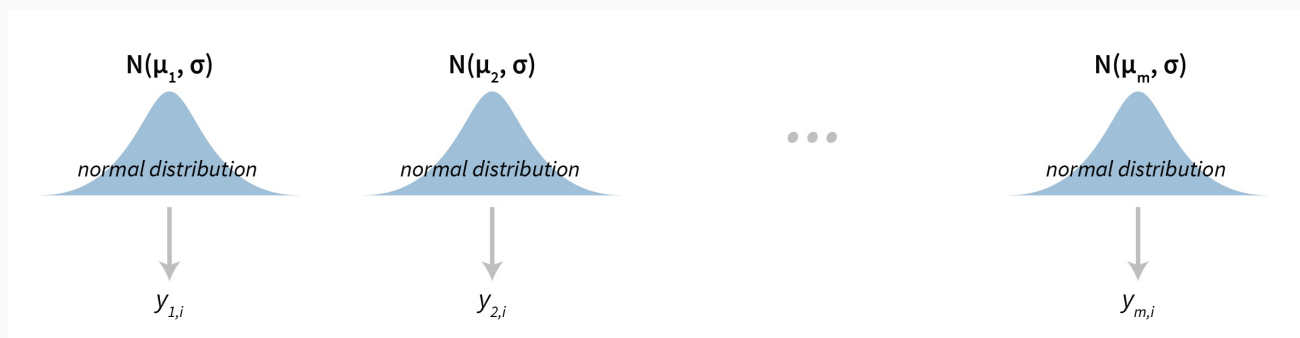
This simple normal model discards the information about mama pigs and fits a normal distribution over all piglet weights. With this approach we obviously lose the ability to compare piglet weights between mama pigs. But this does not mean that the model is useless – it is perfectly OK for making several more general comparisons, such as comparing birth weight of pigs with those of baby tigers (if we had another similar dataset containing weights of baby tigers that is). The figure below visualizes how this model fits the underlying data:



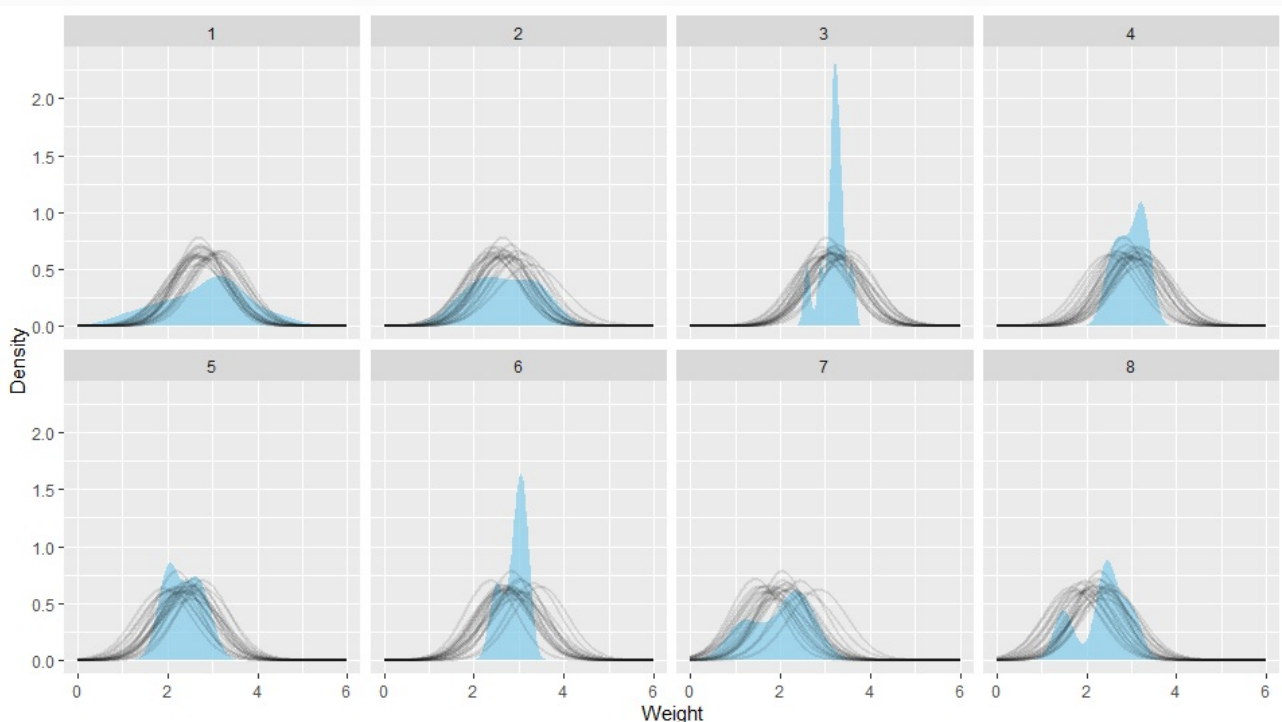
If our goal is not to perform general comparisons but the comparison of piglet birth weights between different mama pigs, then we need a different model. We can simply model the mean of each mama pig (subject) separately:

$$y_{s,i} \mid \mu_s, \sigma \sim N(\mu_s, \sigma),$$

where $y_{s,i}$ represents the weight of piglet i that was born to mama pig s , meaning that μ_s is the mean parameter for subject (mama pig) s . Note here, that we are assuming equal variance (or standard deviation) in weights between all subjects, if we assumed that each subject has its own variance then we could just use the traditional normal model multiple times, once for each mama pig. Below is a visual representation of this model:



Above model acknowledges the fact that each piglet has its own mother, but we discard the fact that they are all piglets. As a result we are now able to make comparisons between subjects (between piglets belonging to different mama pigs) but lose the capability of more general (group level) comparisons (e.g., inter-species). The figure below visualizes how this model fits the underlying for each mama pig (subject) separately:



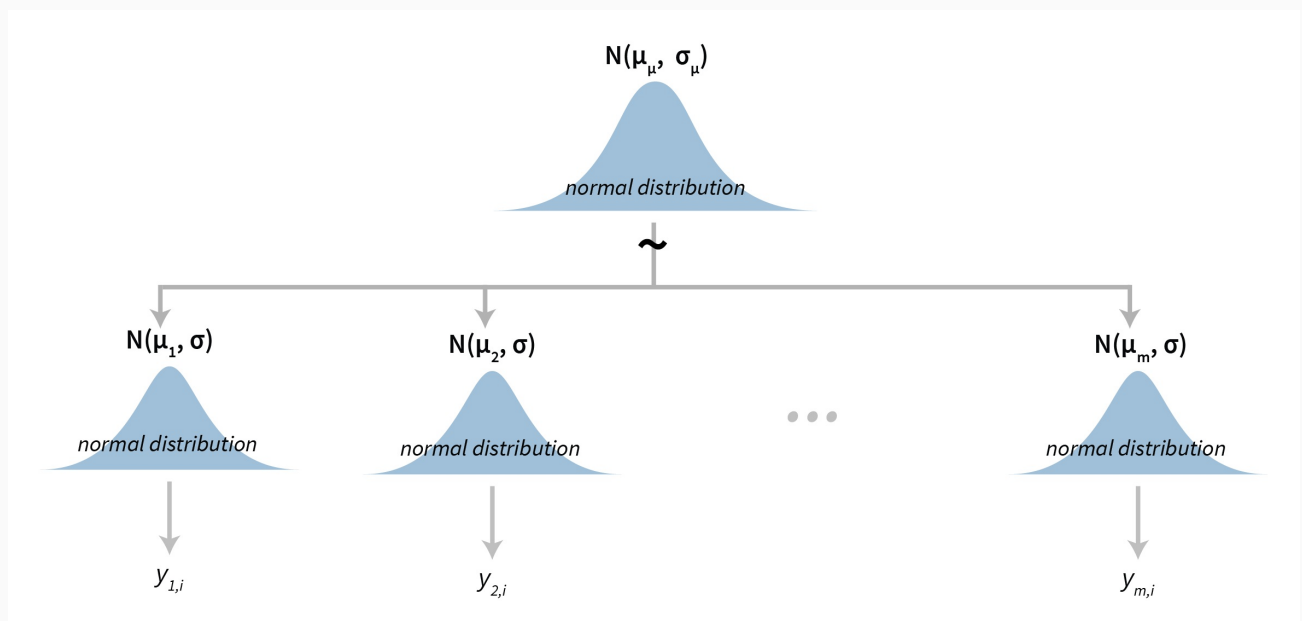
Based on this figure, one could argue that the assumption that the variance between subjects is the same is not correct. This is a valid case, since we can see that because of this constraint our model does not fit the underlying data well in some cases (e.g., mama pig #3 or #6). So, a possible model upgrade would be to also model variances on the subject level. We decided not to do that because we were interested mainly in mean weights and wanted to keep the model as simple as possible for the purpose of this illustration. Furthermore, our dataset is quite small, for getting good variance estimates we usually need more data than we have in this case.

In Bayesian statistics we can easily have the best of both worlds through hierarchical modeling. Actually, one could easily argue that in our case such a model is the only reasonable approach since we have a very clear hierarchical structure in our data generating process. By linking subject means in a hierarchical fashion, our model becomes:

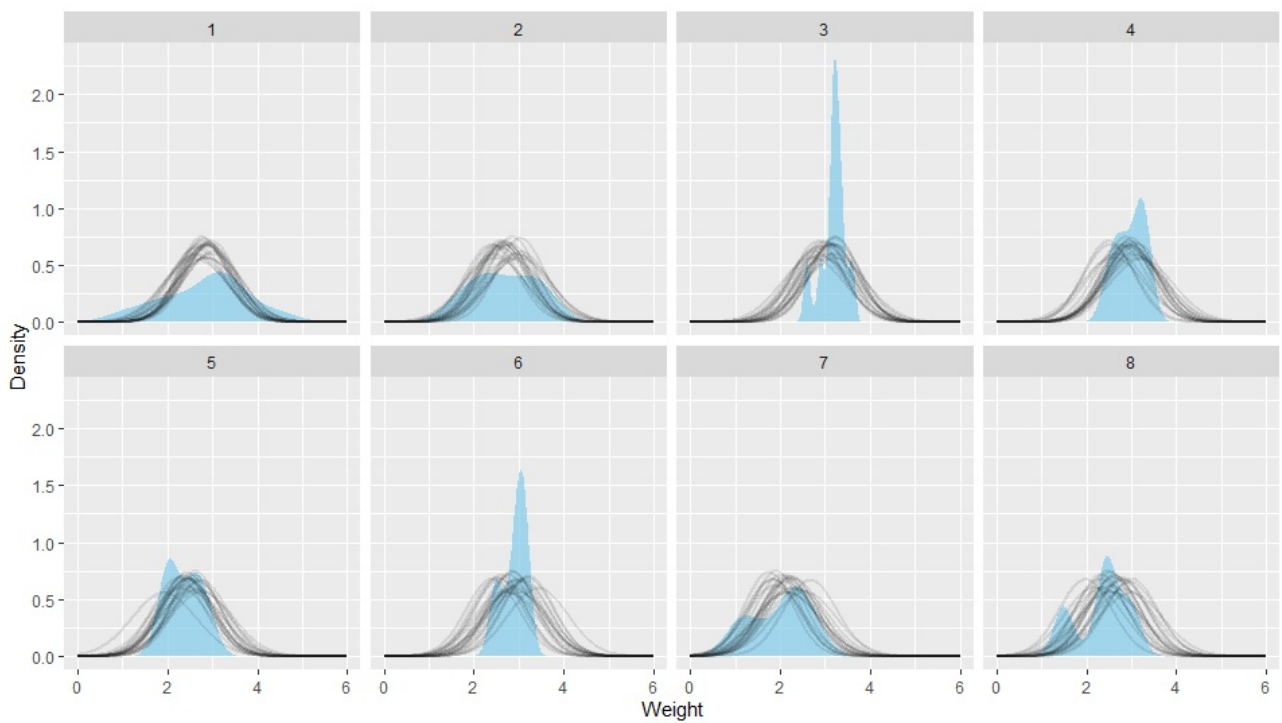
$$y_{s,i} \mid \mu_s, \sigma \sim N(\mu_s, \sigma),$$

$$\mu_s \mid \mu_\mu, \sigma_\mu \sim N(\mu_\mu, \sigma_\mu).$$

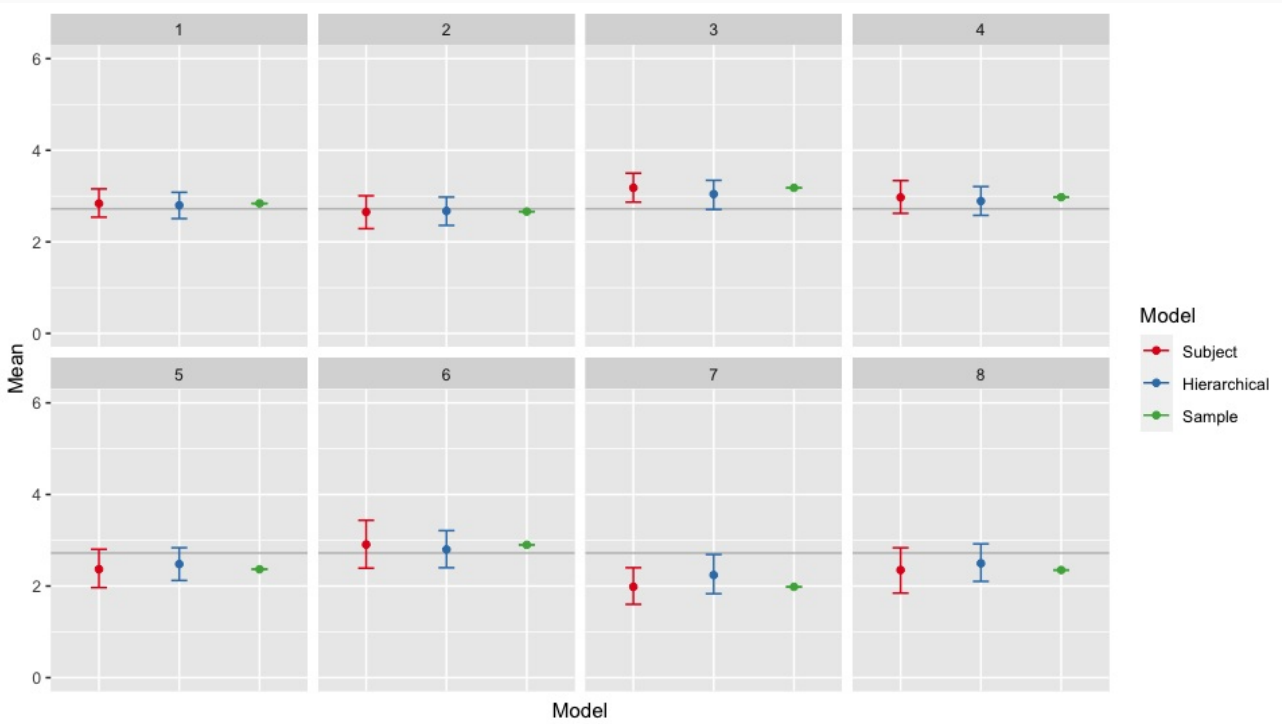
So, to get a hierarchical model we simply inject the information about the hierarchical structure of our data generating process into the model by linking all group level means (μ_s) together using another distribution. In our case we are linking subject level means together with a normal distribution, this top level (group level) normal distribution is defined with parameters μ_μ and σ_μ . As a result μ_μ now defines the mean birth weight of all piglets, while μ_s is the mean birth weight of piglets belonging to a particular mama pig. Here, μ_μ serves a similar function as μ in the traditional normal model – the mean parameter for birth weight of all piglets. It is important to note here that the same does not hold for σ_μ . σ_μ depicts the standard deviation of subject means (μ_s) and not the standard deviation between birth weights of all piglets like in the traditional normal model. This hierarchical structure can be nicely seen on the model's visualization:



To summarize, we now have both subject level parameters (μ_s, σ) to work with, as well as a group level parameter for mean piglet weight (μ_μ) along with its standard deviation (σ_μ). The figure below visualizes how this model fits the underlying data at the subject level.



The figure is very similar to the one made by the previous model (the non-hierarchical model with multiple means). There are a couple of subtle but important differences between the two figures. This differences are even more prominent on the visualization below, which compares means and 90% HDI between both models and our sample means.

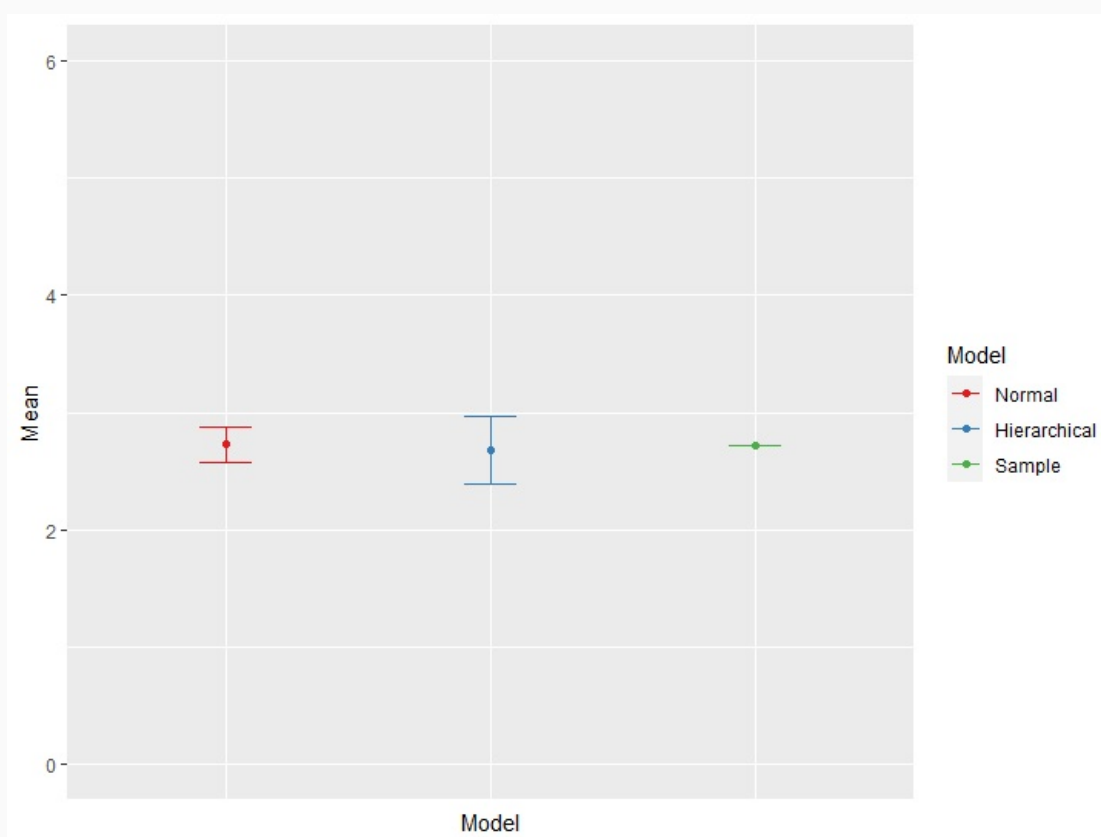


The most important thing to notice is the fact that mean estimates for subjects in the case of the hierarchical model are closer to the population mean (the gray horizontal line) than the mean estimates for subjects from the non-hierarchical model. This phenomenon is called Bayesian shrinkage and is a desired effect in hierarchical models (if we do not want this, then we do not use a hierarchical model). Bayesian shrinkage is the result of model's information pooling or sharing, in hierarchical modeling information about one subject influences inferences for all other subjects. This way subjects with very little data are empowered by all other subjects. This effect will be more prominent in outliers, if one subject sticks out from the rest then our model will be correctly skeptical about its data and will nudge the inferences about it towards other subjects. Indeed, in our case you see it best with mama pigs #3 and #7, these are the two mamas that gave birth to the fattest and the slimmest piglets, and with those two the difference in parameter estimates between

the non-hierarchical and the hierarchical normal models is the most prominent.

Another thing you can see on the image above is that the subject level uncertainty in the case of a hierarchical normal model is lower than in the case of a non-hierarchical normal model. The reason behind this is the same, more information means less uncertainty – in the hierarchical normal model we are using information from other subjects, while in the non-hierarchical normal model we are not doing so.

There are some key differences when comparing models on the group level as well. First, the uncertainty is higher in the case of our hierarchical model. In the hierarchical model we have two sources of uncertainty – we have uncertainty in the subject level means along with uncertainty in the the group level mean (mean of group level means). Even though information is pooled between subjects the overall uncertainty is still higher than in the case of a simple normal model. There we are using all data to estimate a single mean parameter. You can also see that because of the hierarchical structure the estimate for the group level mean is slightly different between two models.



We can now easily fit the hierarchical model to the data from the second farm as well. We can use samples from both hierarchical models (the one fit on pigs from farm 1 and the one fit on pigs from farm 2) to answer several interesting questions. For example, what is the probability that mamas on farm 2 give birth to heavier piglets in comparison to mamas on farm 1? Or we can use the models to find out the mama that gives birth to the heaviest piglets. In other words, we can use the same model to perform analyses both on the subject level (mama pigs) or on the group level (farms). Code for answering the two questions about can be found in the code examples folder.

/ The adaptation level experiment

In the previous example we used a normal distribution to establish a hierarchical structure between groups of normally distributed data. But this does not have to be the case, the building blocks of any hierarchical model do not have to be normally distributed nor do we have to use a normal distribution to link these pieces together. In the following example we will build a hierarchical model based on simple linear regression models.

In this example we will analyze data from an adaptation level experiment. In it participants had to assess weights of the objects placed in their hands by using a verbal scale: very very light, very light, light, medium light, medium, medium heavy, heavy, very heavy and very very heavy. The task was to assess the weight of an object that was placed on the palm of their hand. To standardize the procedure the participants had to place the elbow on the desk, extend the palm and assess the weight of the object after it was placed on their palm by slight up and down movements of their arm. During the experiment participants were blinded by using non-transparent fabric. In total there were 15 objects of the same shape and size but different mass (photo film canisters filled with metallic balls). Objects were grouped into three sets:

- light set: 45 g, 55 g, 65 g, 75 g, 85 g (weights 1 to 5),
- medium set: 95 g, 105 g, 115 g, 125 g, 135 g (weights 6 to 10),
- heavy set: 145 g, 155 g, 165 g, 175 g, 185 g (weights 11 to 15).

The experimenter sequentially placed weights in the palm of the participant and recorded the trial index, the weight of the object and participant's response. The participants were divided into two groups. In group 1, the participants first assessed the weights of the light set in ten rounds within which all five weights were weighted in a random order. After completing the 10 rounds with the light set, the experimenter switched to the medium set, without any announcement or break. The participant then weighted the medium set across another 10 rounds of weighting the five weights in a random order within each round. In group 2, the overall procedure was the same, the only difference being that they started with the 10 rounds of the heavy set and then performed another 10 rounds of weighting of the medium set. Importantly, the weights within each set were given in random order and the experimenter switched between sets seamlessly without any break or other indication to the participant.

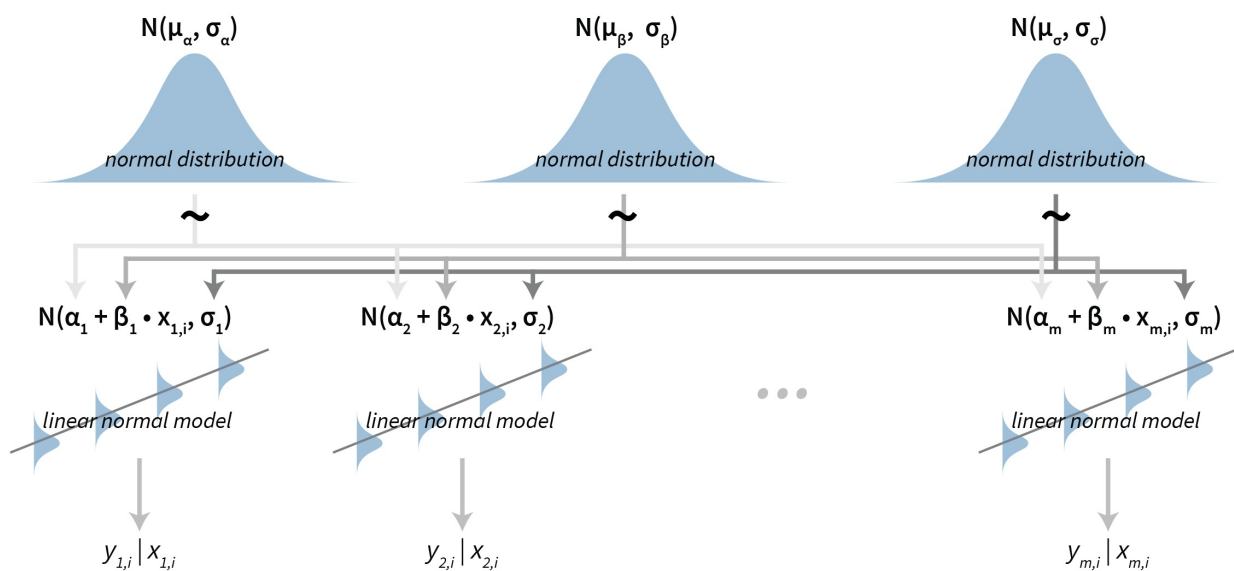
We will use the a hierarchical linear model (`./models/hierarchical_linear.stan`) to show that the two groups provide different assessment of the weights in the second part of the experiment even though both groups are responding to weights from the same (medium) set. The difference is very pronounced at first but then fades away with subsequent assessments of medium weights. This is congruent with the hypothesis that each group formed a different adaptation level during the initial phase of the task, the formed adaptation level then determined the perceptual experience of the same set of weights at the beginning of the second part of the task. In the formal definition and in the figure below, $y_{s,i}$ denotes the response of subject s for round i :

$$y_{s,i} \mid \alpha_s, \beta_s, \sigma_s, i \sim N(\alpha_s + \beta_s i, \sigma_s),$$

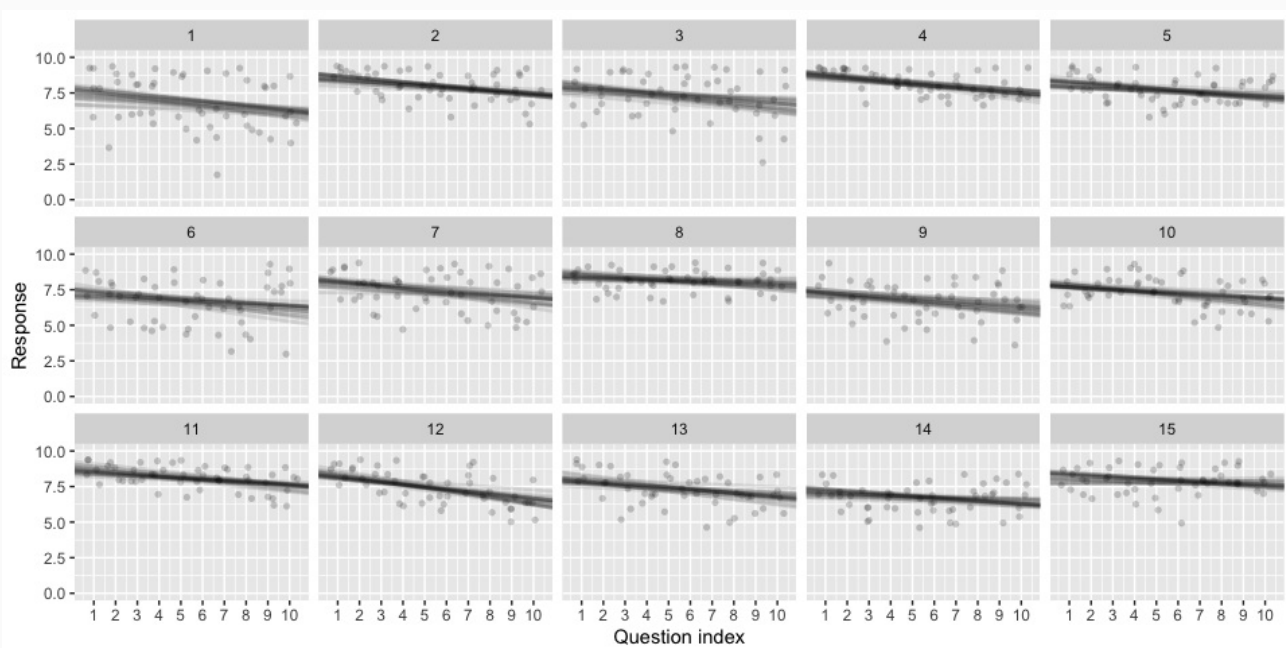
$$\alpha_s \mid \mu_\alpha, \sigma_\alpha \sim N(\mu_\alpha, \sigma_\alpha),$$

$$\beta_s \mid \mu_\beta, \sigma_\beta \sim N(\mu_\beta, \sigma_\beta),$$

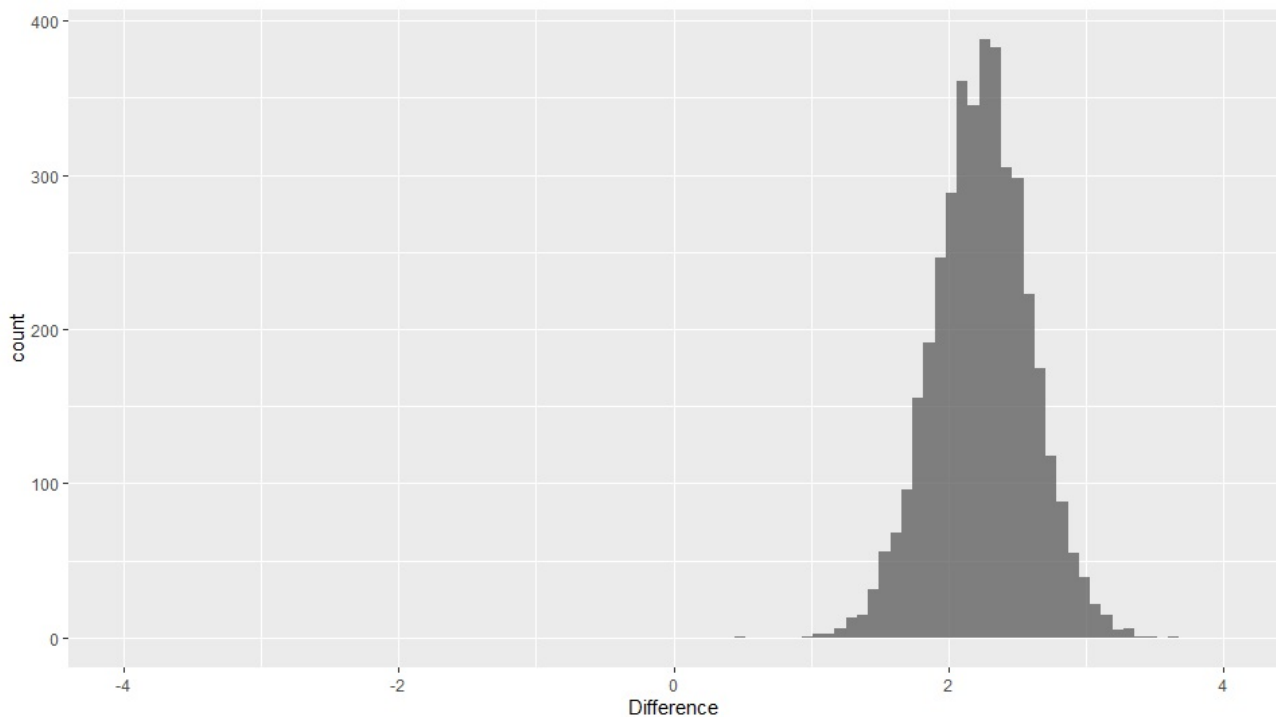
$$\sigma_s \mid \mu_\sigma, \sigma_\sigma \sim N(\mu_\sigma, \sigma_\sigma).$$



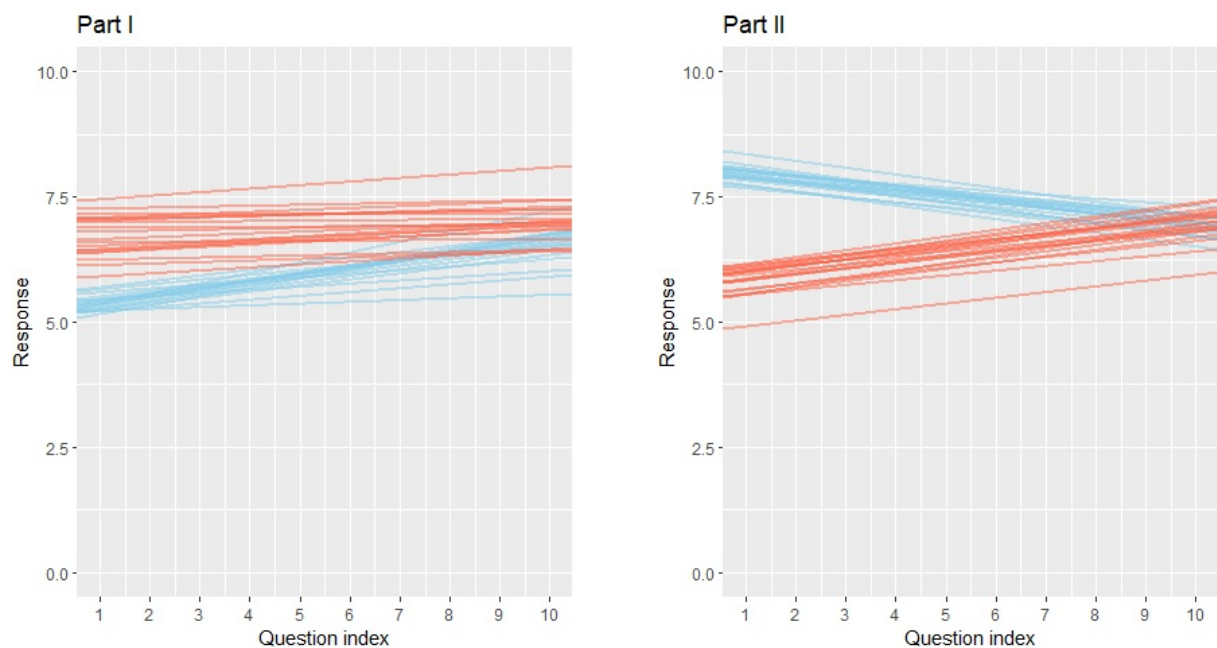
We fit four separate models, one for each combination of group and part of the experiment. Traceplots and summary diagnostics look good so we can move on with our analysis (you can check yourself in [./code/hierarchical_linear.R](#)). Next, we can visually check the quality of our posterior fit. Figure below visualizes the fit for one combination out of four – first group, second part of the experiment.



Values of slope and intercept for the second part of our experiment suggest that our initial hypothesis about adaptation level is true. Subject's that weighted lighter object in the first part of the experiment find medium objects at the beginning of experiment's second part heavier than subjects that weighted heavier objects in the first part. In more technical terms, the intercept for the first group in the second part of the experiment is larger than the one for the second group. See the figure below for a visualization of this.



The fact that the slope for the first group is very likely to be negative (the whole 95% HDI lies below 0) and positive for the second group (the whole 95% HDI lies above 0) suggests that the adaptation level phenomenon fades away with time. This is visualized in the right panel of the figure below.



Based on the analysis above, the hypothesis that each group formed a different adaptation level during the initial phase of the task seems to be true. Group that switches from heavy to medium weights assesses weights as lighter than they really are while for the group that switches from light to medium the weights appear heavier. With time these adaptation levels fade away and assessments converge to similar estimates of weight.

/ Regularization

A Bayesian approach can also overfit the training data, although it is less likely to do so, because it uses distributions over parameters instead of point estimates. In this section we will get familiar with two ways of incorporating regularization into Bayesian models.

// Using a regularizing (skeptical) prior

Bayesian learning combines prior knowledge with data (facts) to update model's beliefs about the data generating process. If priors are completely uninformative (e.g., $U(-\infty, \infty)$) then updated beliefs will be based completely on the input data which can lead to overfitting. To prevent this we can put a more strict prior on model's of our parameter. This way the model will be less excited by the input data. In a way, priors make the model skeptical about the input data. Since such priors achieve the effect of regularization, we often call them regularization priors.

As you already saw by now, we commonly put such priors on β coefficients in our regression models, now you know that the main goal of this is to avoid overfitting. But be careful here, if we are too skeptical (we provide too strict priors) then our model will not pay enough attention to the input data and will underfit! If we standardize our input variables then β coefficients should be close to 0. In our previous models we put a $N(0, 1)$ (or similar) prior on our β coefficients. Below is a more flexible way, in this example of a multiple linear regression model we put a $N(0, \sigma_\beta)$ prior on our β coefficients, where σ_β is a hyper-parameter that we can provide as input to the model. Finding the optimal value of this parameter is usually done in the same way as with hyper-parameters in any other machine learning model – with cross validation or some other approach to model parameter selection.

```
data {
  int<lower=0> n;           // train set size
  int<lower=0> m;           // test set size
  int<lower=0> k;           // number of independent variables
  matrix[n, k] X;          // train set
  matrix[m, k] X_test;     // test set
  vector[n] y;             // dependent variable
  real<lower=0> sigma_beta; // regularization parameter
}

parameters {
  real alpha;              // intercept
  vector[k] beta;          // beta coefficients
  real<lower=0> sigma;      // sd
}

model {
  // regularization
  beta ~ normal(0, sigma_beta);

  y ~ normal(alpha + X * beta, sigma);
}
```

Regularizing priors are not only helpful to prevent overfitting, but can also help out the sampler by narrowing down plausible ranges of parameters, which usually results in a faster and more stable sampling process.

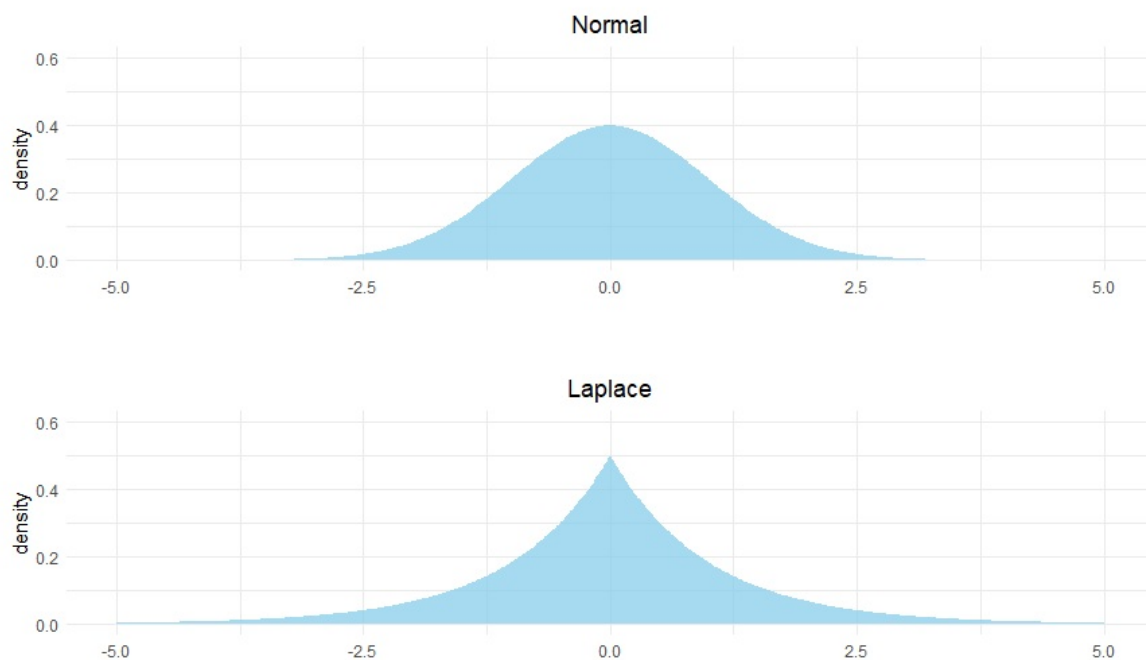
// Using a hyper-prior

By using a hyper-prior we jointly estimate the σ_β parameter along with our β coefficients. We can think of this as a hierarchical Bayesian model (more about those in one of the future lectures) where σ_β is interpreted as a group-level scaling parameters that is estimated from pooled information across individual β coefficients. In other words, β coefficients are linked together through a shared hyper parameter σ_β which can be interpreted as a “penalty” term. If some β coefficients are close to

0, this will pull others towards 0 as well (through the σ_β parameter). We will encounter this phenomena several times, it is quite common and even has its own name – Bayesian shrinkage. So, if σ_β is close to zero then all β coefficients will be shrunk towards 0 and as $\sigma_\beta \rightarrow \infty$ our prior on β becomes uniform.

```
data {  
  int<lower=0> n;      // train set size  
  int<lower=0> m;      // test set size  
  int<lower=0> k;      // number of independent variables  
  matrix[n, k] X;     // train set  
  matrix[m, k] X_test; // test set  
  vector[n] y;        // dependent variable  
}  
  
parameters {  
  real alpha;          // intercept  
  vector[k] beta;      // beta coefficients  
  real<lower=0> sigma;  // sd  
  real<lower=0> sigma_beta; // hierarchical sd across betas  
}  
  
model {  
  // penalized regression - Bayesian L2  
  // per Erp et al. 2019 - Shrinkage priors for Bayesian penalized regression  
  sigma_beta ~ cauchy(0, 1);  
  beta ~ normal(0, sigma_beta);  
  
  // for Bayesian L1 use Laplace or double_exponential prior  
  //beta ~ double_exponential(0, sigma_beta);  
  
  y ~ normal(alpha + X * beta, sigma);  
}
```

The figure below visualizes the normal and Laplace distributions which are used as β priors in the case of the Bayesian L1 and L2 regression. In this case the spread parameter for both was set to 1.



// Comparison

To compare various regularization techniques we will use multiple linear regression on an air pollution dataset (`data/ozone.csv`). We will use 300 independent variables to estimate tomorrow's ozone level (O^3). Our dataset has 1353 observations, we used 500 of them for the training set and the rest for the test set. The split is intentionally skewed in order to emphasize the problem of overfitting.

To evaluate Bayesian regression models we compared them with a popular machine learning model (random forests) and three linear regression models. Traditional linear regression does not incorporate any regularization and thus its loss function does not have a penalty term:

$$\operatorname{argmax}_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^m \beta_j x_{ij})^2.$$

Lasso regression (L1) uses a loss function that includes a penalty term for model complexity, the penalty term is based on absolute values of β coefficients:

$$\operatorname{argmax}_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^m \beta_j x_{ij})^2 + \lambda \sum_{j=1}^m |\beta_j|.$$

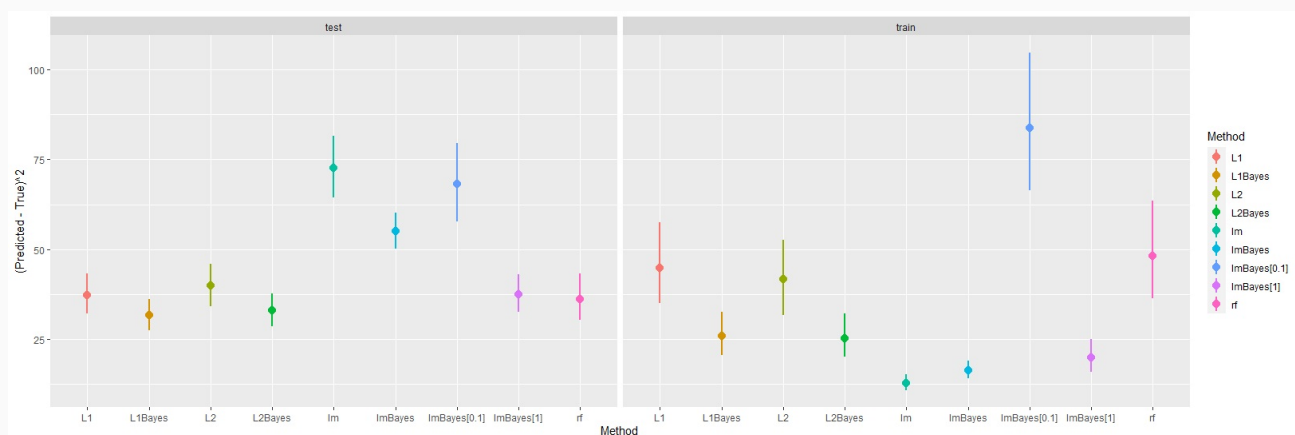
Similarly, ridge regression (L2) facilitates a penalty term that is based on squared β coefficients:

$$\operatorname{argmax}_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^m \beta_j x_{ij})^2 + \lambda \sum_{j=1}^m \beta_j^2.$$

On the Bayesian side our comparison implements five different Bayesian linear regression models:

- Bayesian regression (without regularization, `models/regularization_none.stan`),
- Bayesian regression with a regularizing prior ($\sigma_{\beta} = 0.1$, `models/regularization_parameter.stan`),
- Bayesian regression with a regularizing prior ($\sigma_{\beta} = 1$, `models/regularization_parameter.stan`),
- Bayesian lasso regression (regularization through a hyper-prior, `models/regularization_hyperprior_l1.stan`),
- Bayesian ridge regression (regularization through a hyper-prior, `models/regularization_hyperprior_l2.stan`).

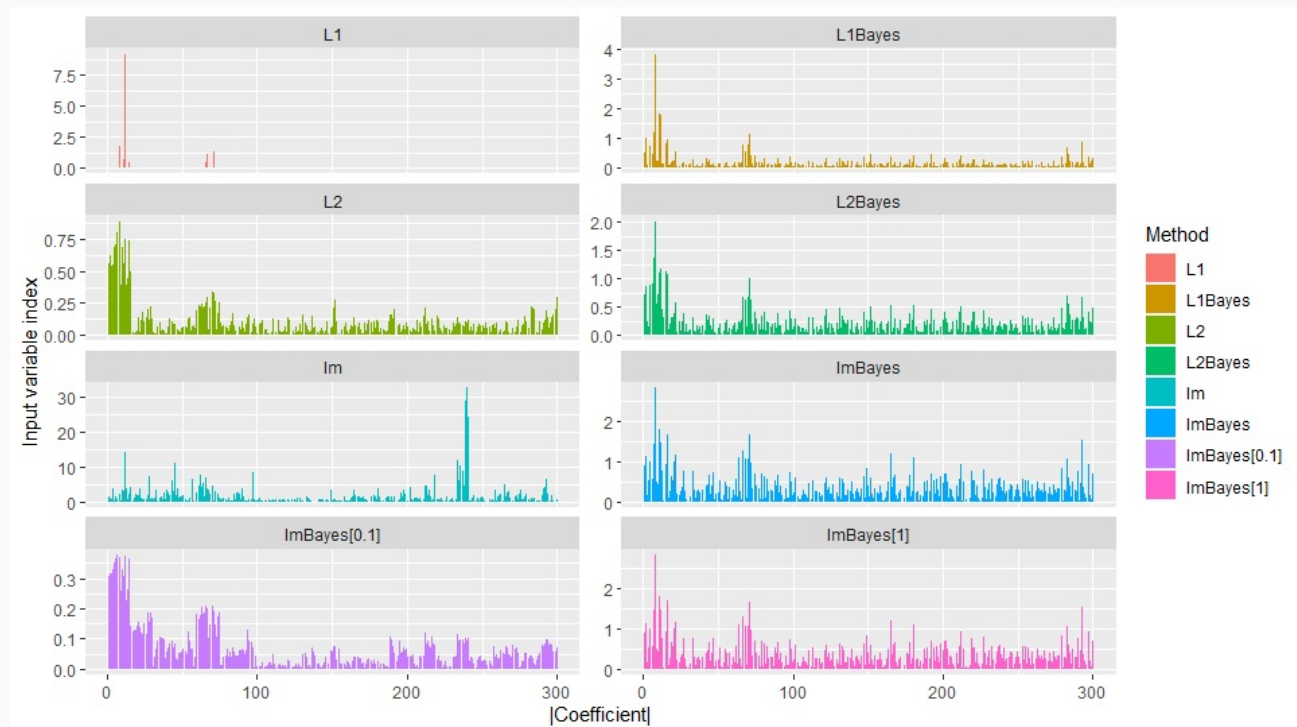
The figure below visualizes results of the comparison.



We can see that when the model has many β coefficients it will overfit if no regularization is implemented. If we are using a too strict regularizing prior (`lmBayes[0.1]`) then our model underfits (the model's prior knowledge is very strict and new facts do not change it too much). We

can see that the best results were achieved when we unleashed the full power of Bayesian modeling by putting a hyper-prior distribution on the spread parameter of β coefficients.

The figure below compares absolute values of β coefficients between all of used regression models.



/ Recommended readings

- **Chapter 12 (Multilevel models)** in McElreath R. (2016). Statistical Rethinking: A Bayesian Course with Examples in R and Stan. CRC Press.
- **Chapter 19 (Metric Predicted Variables with One Nominal Predictor)** in Kruschke J. K. (2015). *Doing Bayesian Data Analysis: A Tutorial with R, JAGS, and Stan* (2nd edition). Academic Press.